Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	**COMPUTER ORGANIZATION AND ARCHITECTURE**
SUBJECT CODE:	**SECR/SCSR 1033**
SEMESTER:	**2 – 2025/2026**
LAB TITLE:	**Lab 2: Arithmetic Equations & Operations**
STUDENT INFO :	Execute the lab in group of two. **Student 1**: *No. 1, 3, 5* **Student 2**: *No 2, 4, 6* **Name 1: NG XUAN YEE** **Metric No.: A25CS0291** **Name 2: CHENG ZHI MIN** **Metric No.: A25CS0050**
SUBMISSION DATE & ITEMS:	**Duration of submission: -** 2 weeks **Submission items in e-learning: -** 1. Lab 2 exercise sheet/file (in .pdf), with the links for demo video 5 - 10min, on the cover page. 3. The assembly programs (in .asm).

MARKS: _____

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine the program's output by inspecting the contents of the relevant registers.

a) Fill in **Table 1** with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

 .code
main PROC
      mov ax, var1 ; LINE1
      mov bx, var2 ; LINE2
      xchg ax, bx  ; LINE3
      mov var1, ax ; LINE4
      mov var2, bx ; LINE5
      call DumpRegs
      exit
main ENDP
END main
```

Answer Q1

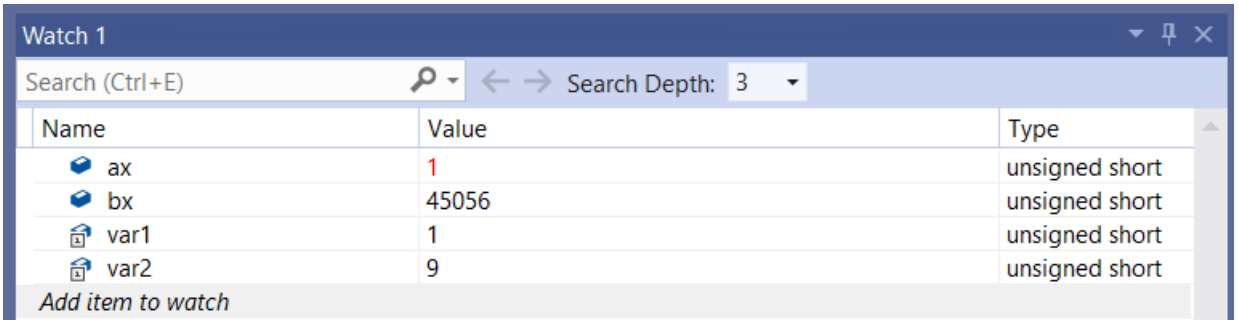
a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	**AX = 001h** **var1 = 0001h**	Move the value of var1 (1) into register AX
LINE2	**BX = 0009h** **var2 = 0009h**	Move the value of var2 (9) into register BX
LINE3	**AX = 0009h** **BX = 0001h**	Exchange the values of AX and BX
LINE4	**AX = 0009h** **var1 = 0009h**	Move the new value of AX into variable var1
LINE5	**BX = 0001h** **Var2 = 0001h**	Move the new value of BX into variable var2

b) Paste here a screenshot of all registers' content after each LINE is executed:

LINE1:



LINE2:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	1	unsigned short
bx	9	unsigned short
var1	1	unsigned short
var2	9	unsigned short
Add item to watch		

LINE3:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	9	unsigned short
bx	1	unsigned short
var1	1	unsigned short
var2	9	unsigned short
Add item to watch		

LINE4:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	9	unsigned short
bx	1	unsigned short
var1	9	unsigned short
var2	9	unsigned short
Add item to watch		

LINE5:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	9	unsigned short
bx	1	unsigned short
var1	9	unsigned short
var2	1	unsigned short
Add item to watch		

Q2. Execute the program below. Determine the program's output by inspecting the contents of the relevant registers and watches.

a) Fill in **Table 2** with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: Rval = (-Xval + (Yval – Zval)) + 1

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
      mov eax,Xval       ; LINE1
      neg eax            ; LINE2
      mov ebx,Yval       ; LINE3
      sub ebx,Zval       ; LINE4
      add eax,ebx        ; LINE5
      inc eax            ; LINE6
      mov Rval,eax       ; LINE7
      exit
main endp
end main
```

Answer Q2

a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	**EAX = 0000001Ah** **Xval = 0000001Ah**	Move the value of Xval (26d) into register EAX
LINE2	**EAX = FFFFFFE6h**	Negate the value in register EAX
LINE3	**EBX = 0000001Eh** **Yval = 0000001Eh**	Move the value of Yval (30) into register EBX
LINE4	**EBX = FFFFFFF6h** **Zval = 00000028h**	Subtract the value of Zval from register EBX
LINE5	**EAX = FFFFFFDCh** **EBX = FFFFFFF6h**	Add the value of register EBX to register EAX
LINE6	**EAX = FFFFFFDDh**	Increament the value in register EAX by 1
LINE7	**EAX = FFFFFFDDh** **Rval = FFFFFFDDh**	Move the value of register EAX (-35) into variable Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	26	unsigned int
ebx	6397952	unsigned int
Rval	0	unsigned long
Xval	26	unsigned long
Yval	30	unsigned long
Zval	40	unsigned long

LINE2:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	4294967270	unsigned int
ebx	6397952	unsigned int
Rval	0	unsigned long
Xval	26	unsigned long
Yval	30	unsigned long
Zval	40	unsigned long

LINE3:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	4294967270	unsigned int
ebx	30	unsigned int
Rval	0	unsigned long
Xval	26	unsigned long
Yval	30	unsigned long
Zval	40	unsigned long

LINE4:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	4294967270	unsigned int
ebx	4294967286	unsigned int
Rval	0	unsigned long
Xval	26	unsigned long
Yval	30	unsigned long
Zval	40	unsigned long

LINE5:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	4294967260	unsigned int
ebx	4294967286	unsigned int
Rval	0	unsigned long
Xval	26	unsigned long
Yval	30	unsigned long
Zval	40	unsigned long

LINE6:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	4294967261	unsigned int
ebx	4294967286	unsigned int
Rval	0	unsigned long
Xval	26	unsigned long
Yval	30	unsigned long
Zval	40	unsigned long

LINE7:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	4294967261	unsigned int
ebx	4294967286	unsigned int
Rval	4294967261	unsigned long
Xval	26	unsigned long
Yval	30	unsigned long
Zval	40	unsigned long

Q3. Execute the program below. Determine the program's output by inspecting the contents of the relevant registers.

a) Fill in **Table 3** with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: var4 = [(var1 * var2) + var3] - 1

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
      mov eax, var1          ; LINE1
      mul var2               ; LINE2
      add eax, var3          ; LINE3
      dec eax                ; LINE4
      exit
main endp
end main
```

Answer Q3

a) Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	**EAX = 00000005h** **var1 = 00000005h**	Move the value of var1 (5d) into register EAX
LINE2	**EAX = 00000032h** **var2 = 0000000Ah**	Multiply the value of var2 (10d) to register EAX
LINE3	**EAX = 00000046h** **var3 = 00000014h**	Add the value of var3 (20d) to register EAX
LINE4	**EAX = 00000045h** **Var4 = 00000045h**	Decrement the value of EAX by 1 and move the value into variable var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	5	unsigned int
var1	5	unsigned long
var2	10	unsigned long
var3	20	unsigned long
var4	0	unsigned long
Add item to watch		

LINE2:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	50	unsigned int
var1	5	unsigned long
var2	10	unsigned long
var3	20	unsigned long
var4	0	unsigned long
Add item to watch		

LINE3:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	70	unsigned int
var1	5	unsigned long
var2	10	unsigned long
var3	20	unsigned long
var4	0	unsigned long
Add item to watch		

LINE4:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
eax	69	unsigned int
var1	5	unsigned long
var2	10	unsigned long
var3	20	unsigned long
var4	69	unsigned long
Add item to watch		

Q4. Execute the program below. Determine the program's output by inspecting the contents of the relevant registers.

a) Fill in **Table 4** with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: var4 = (var1 * 5) / (var2 – 3)

```
include irvine32.inc
.data
     var1 WORD 40
     var2 WORD 10
     var4 WORD ?
.code
main proc
     mov ax,var1      ; LINE1
     mov bx,5         ; LINE2
     mul bx           ; LINE3
     mov bx,var2      ; LINE4
     sub bx,3         ; LINE5
     div bx           ; LINE6
     mov var4,ax      ; LINE7
     exit
main endp
end main
```

Answer Q4

a) Fill (Write) in the contents for the related register in each line: **Table 4**

LINE1	**AX = 0028h** **var1 = 0028h**	Move the value of var1 (40d) into register AX
LINE2	**BX = 0005h**	Move the immediate value (5d) into register BX
LINE3	**AX = 00C8h** **BX = 0005h**	Multiply the value of register BX to register AX and store the value in register DX:AX
LINE4	**BX = 000Ah** **var2 = 000Ah**	Move the value of var2 (10d) into register BX
LINE5	**BX = 0007h**	Subtract the value in register BX with an immediate value (3d)
LINE6	**AX = 001Ch** **BX = 0007h** **DX = 0004h**	Divide the register AX by register BX, the quotient is store in register AX and the remainder is store in register DX
LINE7	**AX = 001Ch** **var4 = 001Ch**	Move the quotient from AX into variable var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	40	unsigned short
bx	53248	unsigned short
dx	4101	unsigned short
var1	40	unsigned short
var2	10	unsigned short
var4	0	unsigned short
Add item to watch		

LINE2:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	40	unsigned short
bx	5	unsigned short
dx	4101	unsigned short
var1	40	unsigned short
var2	10	unsigned short
var4	0	unsigned short
Add item to watch		

LINE3:

Watch 1

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	200	unsigned short
bx	5	unsigned short
dx	0	unsigned short
var1	40	unsigned short
var2	10	unsigned short
var4	0	unsigned short
Add item to watch		

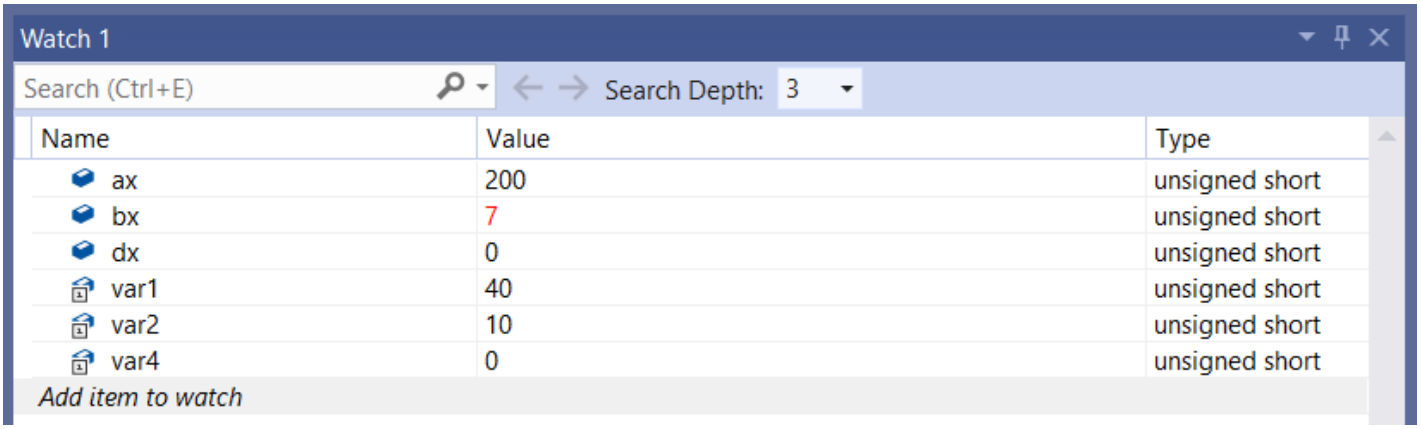
LINE4:

Watch 1

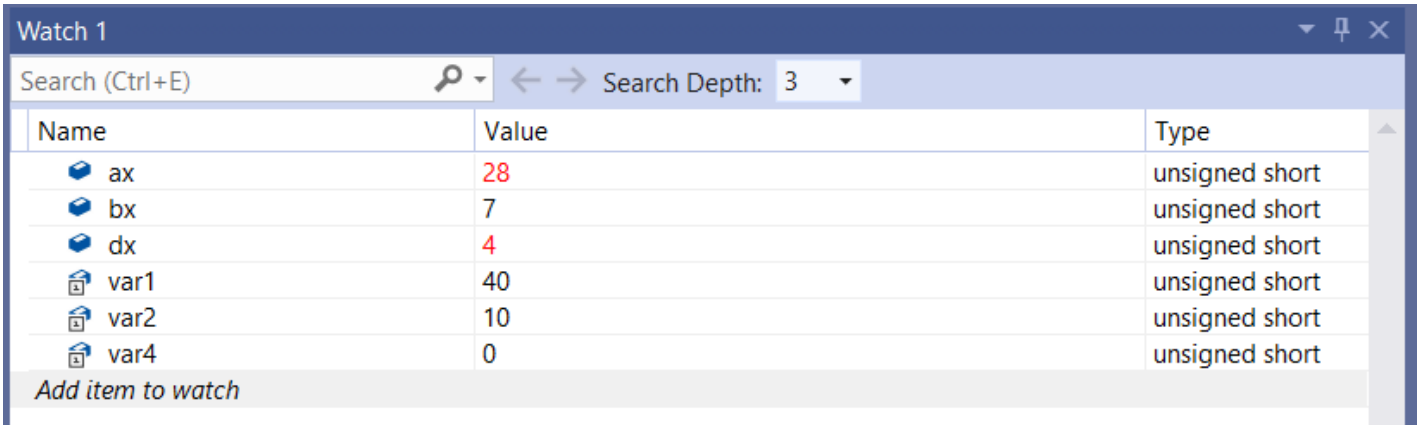
Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	200	unsigned short
bx	10	unsigned short
dx	0	unsigned short
var1	40	unsigned short
var2	10	unsigned short
var4	0	unsigned short
Add item to watch		

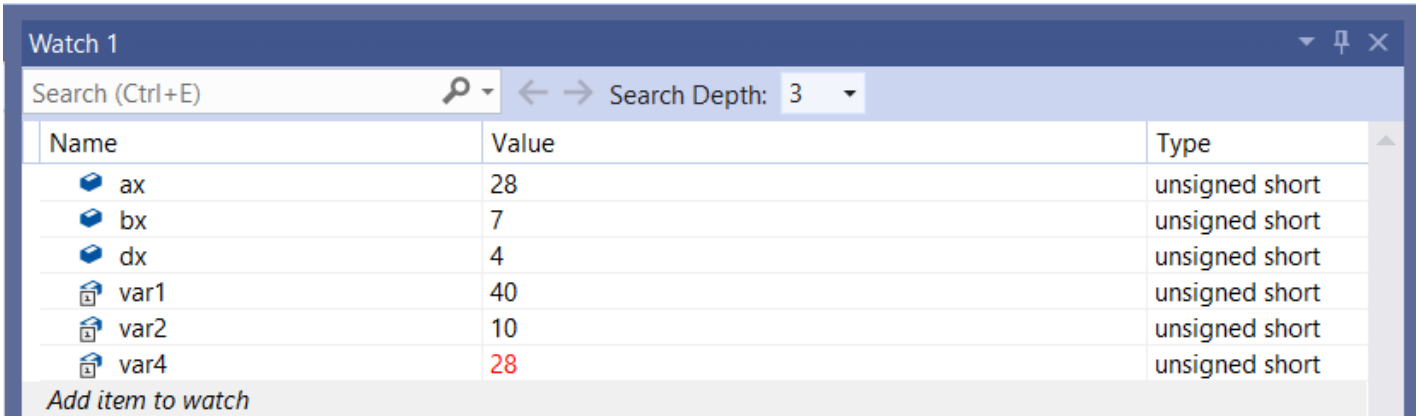
LINE5:



LINE6:



LINE7:



Short Notes for MUL CX and DIV BL:

MUL CX

a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
b. The operand size determines the size of the result:

    i. Byte-sized operand: Result in AX
    ii. Word-sized operand: Result in DX:AX
    iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
    iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX

c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
b. The divisor is specified as the operand of the DIV instruction.
c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
d. The remainder is stored in DX.
e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
f. If the divisor is 0, a division error occurs.
g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.

Q5. Given the following instructions as is, Code Snippet 1.

a) Write a full program to execute Code Snippet 1.

b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX,0        ; Clear DX
MOV AX,1000h    ; Load 1000h into AX
MOV CX, 25h     ; Load 25h into CX
MUL CX          ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

a) Screenshot of full program (.asm) :

main.asm

```
1  TITLE LAB 2 QUESTION 5
2  ; AUthor: NG XUAN YEE & CHENG ZHI MIN
3
4  include Irvine32.inc
5  .code
6  main proc
7      MOV DX,0        ; Clear DX
8      MOV AX,1000h    ; Load 1000h into AX
9      MOV CX, 25h     ; Load 25h into CX
10     MUL CX          ; Multiply AX by CX, storing the result in DX:AX
11     call dumpregs   ; Display the registers
12     exit
13 main endp
14 end main
```

b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=012F5000  EBX=01129000  ECX=00A80025  EDX=00A80002
ESI=00A810AA  EDI=00A810AA  EBP=012FFC34  ESP=012FFC28
EIP=00A83674  EFL=00000A47  CF=1  SF=0  ZF=1  OF=1  AF=0  PF=1
```

Registers

```
EAX = 00DE5000 EBX = 00E4F000 ECX = 00A80025 EDX = 00A80002 ESI = 00A810AA
  EDI = 00A810AA EIP = 00A83674 ESP = 00DEFB54 EBP = 00DEFB60 EFL = 00000A47
```

Q6. Given the following instructions as is, Code Snippet 2.

a) Write a full program to execute Code Snippet 2.

b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0        ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h     ; Load the dividend (8003h) into AX
MOV BL, 10h      ; Load the divisor (10h) into BL
DIV BL           ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

a) Screenshot of full program (.asm) :

main.asm

```
1  TITLE LAB 2 QUESTION 6
2  ; AUthor: NG XUAN YEE & CHENG ZHI MIN
3
4  include Irvine32.inc
5  .code
6  main proc
7      MOV DX, 0        ; Clear DX to form the 16-bit dividend in DX:AX
8      MOV AX, 803h     ; Load the dividend (8003h) into AX
9      MOV BL, 10h      ; Load the divisor (10h) into BL
10     DIV BL           ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
11     call dumpregs    ; Display registers
12     exit
13 main endp
14 end main
```

b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=008F0380  EBX=00641010  ECX=000310AA  EDX=00030000
ESI=000310AA  EDI=000310AA  EBP=008FFEA8  ESP=008FFE9C
EIP=00033671  EFL=00000212  CF=0  SF=0  ZF=0  OF=0  AF=1  PF=0
```

Registers

```
EAX = 00CF0380 EBX = 00AF7010 ECX = 000310AA EDX = 00030000 ESI = 000310AA
  EDI = 000310AA EIP = 00033671 ESP = 00CFF9D8 EBP = 00CFF9E4 EFL = 00000212
```